BÀI TẬP CHƯƠNG 2

Kiến trúc tập lệnh

GV: Nguyễn Thành Nhân
Lớp: IT006.Q21

Họ và tên sinh viên thực hiện: Nguyễn Hoàng Duy Mạnh
MSSV: 25521079

Bài 1.

Phần 1.

a) *f = g + h + i + j;*

b) *f = g + (h + 5);*

g, h, i, j là những số nguyên 32-bit

1.1 Hãy tìm mã hợp ngữ MIPS tương đương các lệnh C trên.

1.2 Có bao nhiêu lệnh MIPS để hiện thực các lệnh C trên.

1.3 Nếu các biến *f, g, h, i* và *j* có giá trị tương ứng là 1, 2, 3, 4, 5 thì giá trị của *f* là bao nhiêu?

Phần 2.

a.

add f, g, h

b.

addi f, f, 1
add f, g, h

1.4 Tìm lệnh C tương đương với các lệnh hợp ngữ MIPS trên.

1.5 Nếu các giá trị *f, g, h* và *i* có giá trị tương ứng 1, 2, 3 và 4 thì giá trị cuối cùng của *f* là bao nhiêu?

Bài 2.

Phần 1.

a. *f = g + h + B[4];*
b. *f = g – A[B[4]];*

f, g, h, i, j được lưu lần lượt ở các thanh ghi *$s0, $s1, $s2, $s3, $s4*

Địa chỉ cơ sở/nền (base address) của mảng A và B được lưu trong các thanh ghi *$s6, $s7*. Mỗi phần

2.1 Hãy chuyển các câu lệnh C bên trên sang dạng hợp ngữ MIPS.

2.2 Cần bao nhiêu lệnh hợp ngữ MIPS để có chức năng tương đương với từng câu lệnh C?

2.3 Có bao nhiêu thanh ghi khác nhau được dùng cho từng câu lệnh C bên trên.

Phần 2.

a.

add $s0, $s0, $s1
add $s0, $s0, $s2
add $s0, $s0, $s3
add $s0, $s0, $s4

b.

lw $s0, 4($s6)

2.4 Hãy tìm câu lệnh C tương đương với các câu lệnh hợp ngữ MIPS bên trên.

2.5 Hãy thử rút gọn số lượng lệnh hợp ngữ MIPS trên nếu có thể.

2.6 Có bao nhiêu thanh ghi được sử dụng trong đoạn hợp ngữ trên? Nếu ta có thể rút gọn số lệnh thì ta cần bao nhiêu thanh ghi?

Bài 3.

a. f = -g + h + B[1];

b. f = A[B[g] + 1];

f, g, h, i, j được lưu lần lượt tại các thanh ghi *$s0, $s1, $s2, $s3, $s4*

Địa chỉ cơ sở của hai chuỗi A và B được lưu trong các thanh ghi *$s6, $s7*

3.1 Hãy tìm các lệnh hợp ngữ MIPS tương đương với các câu lệnh C bên trên.

3.2 Cần bao nhiêu lệnh hợp ngữ MIPS để có chức năng tương đương với từng câu lệnh C?

3.3 Có bao nhiêu thanh ghi khác nhau được dùng cho từng câu lệnh C bên trên.

Bài 4.

Các câu hỏi dưới đây liên quan đến mở rộng dấu và tràn.

Thanh ghi *$s0* và *$s1* lưu các giá trị như bảng bên dưới. Hãy trả lời các câu hỏi liên quan đến lệnh hợp ngữ MIPS bên dưới và tính toán các kết quả.

a. $s0 = 0x70000000; $s1 = 0x0FFFFFFF
b. $s0 = 0x40000000; $s1 = 0x40000000

Ghi chú: Khi 0x trước một giá trị thì giá trị đó đang biểu diễn trong hệ 16

4.1

Tính kết quả của *$t0* sau khi thực hiện câu lệnh: *add $t0, $s0, $s1*

Kết quả trong thanh ghi *$t0* đúng như mong muốn của phép toán chưa? Có xảy ra tràn không?

4.2

Tính kết quả của $t0 sau khi thực hiện câu lệnh: *sub $t0, $s0, $s1*

Kết quả trong thanh ghi $t0 đúng như mong muốn của phép toán chưa? Có xảy ra tràn không?

4.3

Tính kết quả của $t0 sau khi chạy chuỗi lệnh:

add $t0, $s0, $s1

add $t0, $t0, $s0

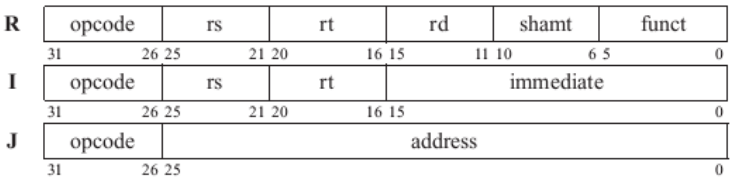
Kết quả trong thanh ghi $t0 đúng như mong muốn của phép toán chưa? Có xảy ra tràn không?

Bài 5.

Chuyển các mã máy sau sang dạng hợp ngữ MIPS

a.	1010 1110 0000 1011 0000 0000 0000 0100_{two}
b.	1000 1101 0000 1000 0000 0000 0100 0000_{two}

5.1 & 5.2. Từ các giá trị binary ở bảng trên, hãy xác định chuỗi nhị phân thể hiện là lệnh gì?
Xác định các lệnh trên là thuộc kiểu lệnh gì (I-type, R-type, J-type).



Chú ý: Tham khảo "MIPS reference data" (trang 2, sách tham khảo) để dò tìm opcode của các lệnh.

5.3
Nếu chuỗi nhị phân trên chỉ là dữ liệu đơn thuần. Hãy chuyển chúng sang dạng mã HEX.

5.4 & 5.5
Hãy dịch các lệnh sau sang dạng mã máy
a) add $t0, $t0, $zero
b) lw $t1, 4($s3)

5.6 Hãy trình bày dưới dạng mã HEX của các trường opcode, Rs và Rt của các lệnh trên. Đối với các lệnh kiểu R, hãy trình bày dưới dạng mã HEX của các trường Rd và funct. Đối với các lệnh kiểu I, hãy trình bày dưới dạng mã HEX trường trực tiếp (immediate field).

Bài 6.
Cho giá trị của các thanh ghi sau:

a.	$t0 = 0x55555555, $t1 = 0x12345678
b.	$t0 = 0xBEADFEED, $t1 = 0xDEADFADE

6.1 Hãy cho biết giá trị của thanh ghi $t2 sau khi chạy các lệnh sau:
sll $t2, $t0, 4
or $t2, $t2, $t1
6.2 Hãy cho biết giá trị của thanh ghi $t2 sau khi chạy các lệnh sau:
srl $t2, $t0, 3
andi $t2, $t2, 0xFFEF

Bài 7.
Giá trị của thanh ghi $t0 được cho trong bảng bên dưới

a.	$1010\ 1101\ 0001\ 0000\ 0000\ 0000\ 0000\ 0010_{two}$
b.	$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two}$

7.1

Giả sử rằng thanh ghi $t0 chứa giá trị ở bảng trên và $t1 có giá trị

$0011\ 1111\ 1111\ 1000\ 0000\ 0000\ 0000\ 0000_{two}$

Hãy cho biết giá trị của $t2 sau khi chạy các lệnh dưới

```
        slt  $t2, $t0, $t1          # set on less than
        beq  $t2, $zero, ELSE       # go to ELSE if $t2=0
        j    DONE                   # go to DONE
ELSE:   addi $t2, $zero, 2          # $t2 = 0+2
DONE:
```

7.2

Giả sử rằng thanh ghi $t0 chứa giá trị trong bảng trên và được so sánh với giá trị X bằng lệnh MIPS bên dưới. Hãy chú ý cấu trúc của lệnh slti. Tìm giá trị của X (nếu có) để $t2 có giá trị là 1.

slti $t2, $t0, X

7.3

Giả sử con trỏ PC đang có giá trị 0x0000 0020.

Ta có thể sử dụng lệnh nhảy trong hợp ngữ MIPS (lệnh j) để nhảy đến địa chỉ có giá trị như trong bảng trên hay không.

Tương tự, ta có thể sử dụng lệnh nhảy-nếu-bằng (lệnh beq) để nhảy đến địa chỉ có trong bảng trên hay không.

Bài 8. Tìm mã máy (biểu diễn hệ 16) cho các lệnh assembly của MIPS sau:

a. *and $t3, $s0, $s2*
b. *sll $t1, $t5, 7*
c. *addi $t0, $s3, 25*
d. *addi $t0, $s3, -25*
e. *lw $t0, 24($s0)*
f. *lw $t0, -24($s0)*
g. *sw $t2, 48($s0)*
h. *sw $t2, -48($s0)*

Bài 9. Cho các mã máy như sau, hỏi tương ứng với từng mã máy là lệnh assembly gì của MIPS

a. *0x01304024*
b. *0x2128fff3 (0x2128FFF3)* ***(Trong hệ 16, các chữ từ a tới f có thể viết thường hoặc hoa đều được)***
c. *0xad28fffc*

Bài 10. Cho đoạn lệnh assembly sau, cho biết kết quả sau khi chạy

a. *and $t0, $s0, $s1*

```
        or $t1, $s0, $s1
        nor $t0, $t0, $t1
        sll $t0, $t0, 3
```

Biết trước khi chạy: *$s0 = 0x12345678; $s1 = 0x00000007*
Hỏi sau khi chạy xong đoạn lệnh trên, *$s0, $s1, $t0, $t1* bằng bao nhiêu

b.
```
        andi $t0, $s0, 12
        nor $t0, $t0, $zero
        ori $t0, $t0, 3
        srl $t0, $t0, 2
```

Biết trước khi chạy: *$s0 = 0x0000000f*
Hỏi sau khi chạy xong đoạn lệnh trên, *$s0, $t0* bằng bao nhiêu

c.
```
        slt     $t2, $t0, $t1
        beq     $t2, $zero, ELSE
        add     $t2, $t2, $t0
        j       DONE
ELSE:   add     $t2, $t2, $t1
DONE:
```

Biết trước khi chạy: *$t0 = 0x0000008f* ; *$t1 = 0x0000009f*
Hỏi sau khi chạy xong đoạn lệnh trên, *$t2* bằng bao nhiêu

d.
```
        addi    $s0, $zero, 2
        addi    $t1, $zero, 6
loop:   beq     $t1, $zero, end
        sll     $s0, $s0, 1
        addi    $t1, $t1, -1
        j       loop
end:    addi    $s1, $s0, 2
```

Sau đoạn chương trình này thì giá trị trong thanh ghi *$s0* là bao nhiêu?

Bài 11. Chuyển các đoạn lệnh C sau sang assembly của MIPS.
Biết *i* và *j* tương ứng với các thanh ghi *$s0* và *$s1*. Mảng A là mảng mà các phần tử là số nguyên, mỗi phần tử chiếm 1 từ nhớ (4 bytes) và địa chỉ nền của mảng A lưu trong thanh ghi *$s3*

a.
```
if (i < j) {
A[i] = A[i] + 1;
A[i+1] = 5;
}else{
A[i] = A[i] - 1;
A[i+1] = 10;
}
i++;
```

b.
```
if (i <= j && j > 0)
```

```
A[j] = A[i] + A[i+1];
else
A[j] = A[i] - A[i+1];
}
i++;
```

c.

```
while (i > 0){
A[i+1] =A[i]*8;
i--;
}
A[0] = 5;
```

d.

```
j = value;
for(i = 1; i < j; i++)
A[i] = B[i];
j = 0;
```

(Với địa chỉ nền mảng B đang lưu trong thanh ghi $s4 và biến value tương ứng thanh ghi $s5)

e.

```
j = value;
max = 0;
for(i = 0; i < j; i++)
if(A[i] > max) max = A[i];
j = 0;
```

(Với biến max tương ứng với thanh ghi $s4)

Câu 4:

```
.data
   input:  .asciiz "Nhap vao 1 chuoi: "
   output: .asciiz "Chuoi sau khi in hoa: "
   buffer: .space 100    # Bo nho dem chua chuoi (toi da 100 ky tu)

.text
main:
        li $v0,4
        la $a0, input
        syscall

        li $v0, 8
        la $a0, buffer
        li $a1, 100
        syscall

        la $t0, buffer
loop:
        # Đến hết mảng thì xuất
        lb $t1, ($t0)
        beqz $t1, outp

        # Nằm trong khoảng 97 – 122 thì giữ lại, sau đó trừ 32 để ra chữ in hoa
        li $t2, 97
        blt $t1, $t2, next_char

        li $t2, 122
        bgt $t1, $t2, next_char

        subi $t1, $t1, 32
        sb $t1, ($t0)
next_char:
        addi $t0,$t0,1
        j loop
outp:
   li $v0, 4
   la $a0, output
   syscall

   li $v0, 4
   la $a0, buffer
   syscall

   li $v0, 10
   syscall
```

Bài 1:
Phần 1:
1.1
a. add $t0, $s1, $s2 # t0 = g + h
add $t1, $s3, $s4 # t1 = i + j
add $s0, $t0, $t1 # f = (g+h) + (i+j)
b. addi $t0, $s2, 5 # t0 = h + 5
add $s0, $s1, $t0 # f = g + t0
1.2.
a. Cần 3 lệnh MIPS
b. Cần 2 lệnh MIPS
1.3.
a. f = 14
b. f = 10
Phần 2:
1.4
a. f = g + h
b. f += 1
f = g + h
1.5
a. f = 5
b. f = 5 (Lệnh cuối là ghi đè f = g + h, nên kết quả phụ thuộc vào g + h)

Bài 2:

2.1

a.
```
lw  $t0, 16($s7)     # t0 = B[4]
add $t1, $s1, $s2    # t1 = g + h
add $s0, $t1, $t0    # f = g + h + B[4]
```

b.
```
lw  $t0, 16($s7)     # t0 = B[4]
sll $t0, $t0, 2      # t0 = B[4] * 4
add $t0, $s6, $t0    # t0 = địa chỉ A[B[4]]
lw  $t1, 0($t0)      # t1 = A[B[4]]
sub $s0, $s1, $t1    # f = g - A[B[4]]
```

2.2

a. Cần 3 lệnh MIPS

b. Cần 5 lệnh MIPS

2.3

a. Cần tổng cộng 6 thanh ghi

b. Cần tổng cộng 6 thanh ghi

2.4

a.
```
add $s0, $s0, $s1
add $s0, $s0, $s2
add $s0, $s0, $s3
add $s0, $s0, $s4
```

b.
```
lw $s0, 4($s6)
```

2.5

a. Có 5 phép cộng, mỗi phép tối đa 2 thanh ghi, vậy cần ít nhất 4 lệnh để thực hiện -> Không rút gọn được nữa

b. 1 lệnh đã là tối thiểu -> Không rút gọn được nữa

2.6

a. Đoạn gốc có 5 thanh ghi tương ứng với 5 biến, đây đã là số thanh ghi tối thiểu

b. Đoạn gốc có 2 thanh ghi, cũng là tối thiểu để lệnh chạy

Bài 3:

3.1

a.
```
sub  $t0, $zero, $s1      # $t0 = -g
lw   $t1, 4($s7)          # $t1 = B[1]
add  $t0, $t0, $s2        # $t0 = -g + h
add  $s0, $t0, $t1        # f = -g + h + B[1]
```

b.
```
sll  $t0, $s1, 2          # $t0 = g * 4
add  $t0, $s7, $t0        # $t0 = địa chỉ B[g]
lw   $t1, 0($t0)          # $t1 = B[g]
addi $t1, $t1, 1          # $t1 = B[g] + 1
sll  $t1, $t1, 2          # $t1 = (B[g] + 1) * 4
add  $t1, $s6, $t1        # $t1 = địa chỉ A[B[g] + 1]
lw   $s0, 0($t1)          # f = A[B[g] + 1]
```

3.2

a. Cần 4 lệnh MIPS

b. Cần 7 lệnh MIPS

3.3.

a. Có 6 thanh ghi

b. Có 6 thanh ghi

Bài 4:

4.1

a. 0x70000000 + 0x0FFFFFFF = 0x7FFFFFFF

0x7FFFFFFF = 2,147,483,647 (Đây là max của số nguyên có dấu 32 bit)

⇨ Không tràn

b.

0x40000000 + 0x40000000=0x80000000

0x80000000 = −2,147,483,648 (Cộng 2 số dương ra 1 số âm)

⇨ Tràn

4.2

a. 0x70000000 − 0x0FFFFFFF = 0x60000001

0x60000001 = 1,610,612,737

⇨ Không tràn

b. 0x40000000 − 0x40000000 = 0x00000000

⇨ Không tràn

4.3

a. Như trên, sau lệnh 1: $t0 = 0x7FFFFFFF

Sau lệnh 2: 0x7FFFFFFF + 0x70000000 = 0xEFFFFFFF (Là số âm)

- Cộng 2 số dương ra 1 số âm => Tràn

b. Như trên, sau lệnh 1: $t0 = 0x80000000 (Đã tràn)

Sau lệnh 2: 0x80000000 + 0x40000000 = $t0=0xC0000000

- Em không chắc sau lệnh 1 chương trình có chạy nữa không, nhưng nếu cộng tiếp thì sau lệnh 2 sẽ ra số không tràn

Bài 5:

5.1 & 5.2

a. 1010 1110 0000 1011 0000 0000 0000 0100
op: 101011 (101011 là lệnh sw, vậy là I-type)
Tách theo định dạng I-type:
rs: 10000 = 16 = $s0
rt: 01011 = 11 = $t3
immediate: 0000000000000100 = 4

Vậy là lệnh: sw $t3, 4($s0)

b. 1000 1101 0000 1000 0000 0000 0100 0000
op: 100011 (101011 là lệnh sw, vậy là I-type)
Tách theo định dạng I-type:
rs: 01000= 8 = $t0
rt: 01000= 8 = $t0
immediate: 0000000001000000 = 64

5.3 Nếu chuỗi trên chỉ là dữ liệu, khi chuyển sang hex sẽ thành:

a. 1010 1110 0000 1011 0000 0000 0000 0100 = 0xAE0B0004

b. 1000 1101 0000 1000 0000 0000 0100 0000 = 0x8E080040

5.4 & 5.5

a. Lệnh add -> R-type -> op = 000000
Cú pháp của lệnh: add $rd, $rs, $rt
rd = $t0 = 8 = 01000
rs = $t0 = 8 = 01000
rt = $zero = 0 = 00000
shamt = 00000
funct = 100000 (add có funct = 32 = 0x20)
=> 000000 01000 00000 01000 00000 100000

b. Lệnh lw -> I-type -> op = 100011
Cú pháp của lệnh: lw rt, immediate(rs)
rt = $t1 = 9 = 01001
rs = $s3 = 19 = 10011
immediate = 4 = 0000000000000100
=> 100011 10011 01001 0000000000000100

5.6

a. 000000 01000 00000 01000 00000 100000
opcode = 0x00
rs = 0x08
rt = 0x00
rd = 0x08
shamt = 0x00
funct = 0x20

b. 100011 10011 01001 0000000000000100
op = 0x23
rs = 0x13
rt = 0x09
immediate = 0x0004

Bài 6:

6.1

a. 0x55555555 << 4=0x55555550
0x55555550 | 0x12345678 = 0x57755778

b. 0xBEADFEED << 4 = 0xEADFEED0
0xEADFEED0 | 0xDEADFADE=0xFEFFFEDE

6.2

a. 0x55555555>>3=0x0AAAAAAA
0x0AAAAAAA & 0x0000FFEF=0x0000AAAA

b. 0xBEADFEED>>3=0x17D5BFDD
0x17D5BFDD & 0x0000FFEF=0x0000BFCD

Bài 7:
7.1

$t1 = 0011,1111,1111,1000,0000,0000,0000,0000_2 = 0x3FF80000$
Đoạn lệnh:
slt $t2, $t0, $t1
beq $t2, $zero, ELSE
j DONE
ELSE: addi $t2, $zero, 2
DONE:
Ý nghĩa:

if ($t0 < $t1) $t2 = 1
else $t2 = 2

Trường hợp	$t0	So sánh	$t2
a	0xAD100002 (số âm)	$t0 < t1$	1
b	0xFFFFFFFF = -1	$t0 < t1$	1

Đáp án:
a) $t2 = 1
b) $t2 = 1

7.2
Lệnh:
slti $t2, $t0, X
$t2 = 1$ khi:
$t0 < X
Giới hạn của X (16-bit signed):
−32768 <= X <= 32767

a)
$t0 = 0xAD100002$ (số âm rất nhỏ)
-1391452158 < X
-> mọi giá trị X hợp lệ đều thỏa
Kết quả:
−32768 <= X <= 32767

7.3
Cho:
PC=0x00000020
Lệnh j
Địa chỉ đích phải:
+ có 4 bit cao giống (PC+4)
+ 2 bit cuối = 00

Các địa chỉ trong bảng:

0xAD100002
0xFFFFFFFF
→ không thỏa
Không nhảy được bằng j.

Lệnh beq
Địa chỉ đích:

PC+4+(offset<<2)
→ luôn có 2 bit cuối = 00
Hai địa chỉ đề bài:
...02
...FF
→ không chia hết cho 4
Không nhảy được bằng beq.

Bài 8:

$s0 = 16 = 10000
$s2 = 18 = 10010
$s3 = 19 = 10011
$t0 = 8 = 01000
$t1 = 9 = 01001
$t2 = 10 = 01010
$t3 = 11 = 01011
$t5 = 13 = 01101
and là R-type: opcode = 0, funct = 36 = 0x24
sll là R-type: opcode = 0, funct = 0
addi: opcode = 8 = 0x08
lw: opcode = 35 = 0x23
sw: opcode = 43 = 0x2B

a. and $t3, $s0, $s2 → 00000010000100100101100000100100 → 0x02125824
b. sll $t1, $t5, 7 → 00000000000011010100100111000000 → 0x000D49C0
c. addi $t0, $s3, 25 → 00100010011010000000000000011001 → 0x22680019
d. addi $t0, $s3, -25 → 00100010011010001111111111100111 → 0x2268FFE7
e. lw $t0, 24($s0) → 10001110000010000000000000011000 → 0x8E080018
f. lw $t0, -24($s0) → 10001110000010001111111111101000 → 0x8E08FFE8
g. sw $t2, 48($s0) → 10101110000010100000000000110000 → 0xAE0A0030
h. sw $t2, -48($s0) → 10101110000010101111111111010000 → 0xAE0AFFD0

Bài 9:

a. 0x01304024 = 00000001001100000100000000100100

opcode = 000000 - > R-type:

opcode = 000000

rs = 01001 -> $t1

rt = 10000 -> $s0

rd = 01000 -> $t0

shamt = 00000

funct = 100100 -> and

-> and $t0, $t1, $s0

b. 0x2128FFF3

= 00100001001010001111111111110011

opcode = 001000 -> addi

rs = 01001 -> $t1

rt = 01000 -> $t0

immediate = 1111111111110011 = -13

-> addi $t0, $t1, -13

c. 0xAD28FFFC = 101011 01001 01000 1111111111111100

opcode = 101011 -> I-type:

opcode = 101011 -> sw

rs = 01001 -> $t1

rt = 01000 -> $t0

immediate = 1111111111111100 = -4

-> sw $t0, -4($t1)

Bài 10:
a.
and $t0, $s0, $s1
or $t1, $s0, $s1
nor $t0, $t0, $t1
sll $t0, $t0, 3
Biết:
$s0 = 0x12345678
$s1 = 0x00000007
Bước 1: and $t0, $s0, $s1
0x12345678 AND 0x00000007
= 0x00000000
Vì 3 bit cuối của 0x12345678 là 000, mà 0x7 = 111.
=> $t0 = 0x00000000
Bước 2: or $t1, $s0, $s1
0x12345678 OR 0x00000007
= 0x1234567F
=> $t1 = 0x1234567F

Bước 3: nor $t0, $t0, $t1
nor = NOT(OR)
$t0 OR $t1 = 0x00000000 OR 0x1234567F = 0x1234567F
NOR => ~0x1234567F = 0xEDCBA980
=> $t0 = 0xEDCBA980

Bước 4: sll $t0, $t0, 3
Dịch trái 3 bit:
0xEDCBA980 << 3 = 0x6E5D4C00
=> $t0 = 0x6E5D4C00
Đáp án:
$s0 = 0x12345678
$s1 = 0x00000007
$t0 = 0x6E5D4C00
$t1 = 0x1234567F

b.
andi $t0, $s0, 12
nor $t0, $t0, $zero
ori $t0, $t0, 3
srl $t0, $t0, 2
Biết:
$s0 = 0x0000000F
Bước 1: andi $t0, $s0, 12
12 thập phân = 0x0000000C
0x0000000F AND 0x0000000C = 0x0000000C
=> $t0 = 0x0000000C
Bước 2: nor $t0, $t0, $zero
$t0 OR $zero = 0x0000000C
NOR => ~0x0000000C = 0xFFFFFFF3
=> $t0 = 0xFFFFFFF3
Bước 3: ori $t0, $t0, 3
0xFFFFFFF3 OR 0x00000003 = 0xFFFFFFF3
Vì 2 bit cuối đã phù hợp rồi.
=> $t0 = 0xFFFFFFF3
Bước 4: srl $t0, $t0, 2
Dịch phải logic 2 bit:
0xFFFFFFF3 >> 2 = 0x3FFFFFFC
=> $t0 = 0x3FFFFFFC
Đáp án:
$s0 = 0x0000000F
$t0 = 0x3FFFFFFC

c.

```
slt  $t2, $t0, $t1
beq  $t2, $zero, ELSE
add  $t2, $t2, $t0
j    DONE
ELSE: add $t2, $t2, $t1
DONE:
```

Biết:

$t0 = 0x0000008F

$t1 = 0x0000009F

Bước 1: slt $t2, $t0, $t1

So sánh có dấu:

0x8F = 143

0x9F = 159

143 < 159 => đúng

=> $t2 = 1

Bước 2: beq $t2, $zero, ELSE

Vì $t2 = 1, không bằng 0, nên không nhảy.

Bước 3: add $t2, $t2, $t0

$t2 = 1 + 0x0000008F = 0x00000090

=> $t2 = 0x00000090

Bước 4: j DONE

Nhảy đến DONE, bỏ qua nhánh ELSE.

Đáp án:

$t2 = 0x00000090

d.

```
addi $s0, $zero, 2
addi $t1, $zero, 6
loop: beq  $t1, $zero, end
sll  $s0, $s0, 1
addi $t1, $t1, -1
j loop
end:  addi $s1, $s0, 2
```

Bước 1:

Khởi tạo
$s0 = 2
$t1 = 6
Trong vòng lặp, mỗi lần:
$s0 <<= 1 tức là nhân 2
$t1 = $t1 – 1

Bước 2: Lặp cho đến khi $t1 = 0, tức là lặp 6 lần.
Tính $s0
Ban đầu:
$s0 = 2
Sau 6 lần nhân 2:
2 × 2^6 = 2 × 64 = 128
=> $s0 = 128 = 0x00000080

Đáp án:

$s0 = 0x00000080

Bài 11:
Quy ước:
i = $s0
j = $s1
A base = $s3
B base = $s4 (ở câu d)
value = $s5
max = $s4 (ở câu e)
và các thanh ghi tạm: $t0, $t1, $t2, ...
Vì A[i] là số nguyên 4 bytes nên địa chỉ phần tử A[i] là: A + i*4

a.
C:
```
if (i < j) {
   A[i] = A[i] + 1;
   A[i+1] = 5;
}else{
   A[i] = A[i] - 1;
   A[i+1] = 10;
}
i++;
```
MIPS:
```
# if (i < j)
slt  $t0, $s0, $s1        # t0 = 1 nếu i < j
beq  $t0, $zero, ELSE_A    # nếu không thì sang else

# THEN: A[i] = A[i] + 1;
sll  $t1, $s0, 2          # t1 = i*4
add  $t1, $s3, $t1        # t1 = &A[i]
lw   $t2, 0($t1)          # t2 = A[i]
addi $t2, $t2, 1          # A[i] + 1
sw   $t2, 0($t1)          # lưu lại A[i]

# A[i+1] = 5;
addi $t3, $t1, 4          # t3 = &A[i+1]
addi $t2, $zero, 5
sw   $t2, 0($t3)

j    ENDIF_A

ELSE_A:
# A[i] = A[i] - 1;
sll  $t1, $s0, 2          # t1 = i*4
add  $t1, $s3, $t1        # t1 = &A[i]
lw   $t2, 0($t1)          # t2 = A[i]
addi $t2, $t2, -1         # A[i] - 1
```

```
sw   $t2, 0($t1)

# A[i+1] = 10;
addi $t3, $t1, 4          # t3 = &A[i+1]
addi $t2, $zero, 10
sw   $t2, 0($t3)

ENDIF_A:
# i++;
addi $s0, $s0, 1
```

b.
C:
```
if (i <= j && j > 0)
   A[j] = A[i] + A[i+1];
else
   A[j] = A[i] - A[i+1];
i++;
```
MIPS:
```
# kiểm tra i <= j
slt  $t0, $s1, $s0        # t0 = 1 nếu j < i
bne  $t0, $zero, ELSE_B    # nếu j < i thì điều kiện sai

# kiểm tra j > 0
slt  $t0, $zero, $s1       # t0 = 1 nếu 0 < j
beq  $t0, $zero, ELSE_B    # nếu j <= 0 thì sang else

# THEN: A[j] = A[i] + A[i+1];

# lấy A[i]
sll  $t1, $s0, 2
add  $t1, $s3, $t1         # t1 = &A[i]
lw   $t2, 0($t1)           # t2 = A[i]

# lấy A[i+1]
lw   $t3, 4($t1)           # t3 = A[i+1]

add  $t4, $t2, $t3         # t4 = A[i] + A[i+1]

# lưu vào A[j]
sll  $t5, $s1, 2
add  $t5, $s3, $t5         # t5 = &A[j]
sw   $t4, 0($t5)

j    ENDIF

ELSE:
# A[j] = A[i] - A[i+1];

# lấy A[i]
sll  $t1, $s0, 2
add  $t1, $s3, $t1
lw   $t2, 0($t1)

# lấy A[i+1]
lw   $t3, 4($t1)
```

```
sub  $t4, $t2, $t3        # t4 = A[i] - A[i+1]

# lưu vào A[j]
sll  $t5, $s1, 2
add  $t5, $s3, $t5
sw   $t4, 0($t5)

ENDIF:
# i++;
addi $s0, $s0, 1
```

c.
C:
```
while (i > 0){
    A[i+1] = A[i]*8;
    i--;
}
A[0] = 5;
```

MIPS:
```
WHILE:
# while (i > 0)
slt  $t0, $zero, $s0       # t0 = 1 nếu 0 < i
beq  $t0, $zero, ENDWHILE_C

# lấy A[i]
sll  $t1, $s0, 2
add  $t1, $s3, $t1         # t1 = &A[i]
lw   $t2, 0($t1)           # t2 = A[i]

# A[i]*8
sll  $t2, $t2, 3           # t2 = A[i]*8

# lưu vào A[i+1]
sw   $t2, 4($t1)           # A[i+1] = t2

# i--
addi $s0, $s0, -1

j    WHILE

ENDWHILE:
# A[0] = 5
addi $t0, $zero, 5
sw   $t0, 0($s3)
```

d.
C:

```
j = value;
for(i = 1; i < j; i++)
    A[i] = B[i];
j = 0;
```

MIPS:

```
# j = value;
add  $s1, $s5, $zero

# i = 1;
addi $s0, $zero, 1

FOR_D:
# i < j ?
slt  $t0, $s0, $s1
beq  $t0, $zero, ENDFOR

# địa chỉ B[i]
sll  $t1, $s0, 2
add  $t2, $s4, $t1        # t2 = &B[i]
lw   $t3, 0($t2)          # t3 = B[i]

# địa chỉ A[i]
add  $t4, $s3, $t1        # t4 = &A[i]
sw   $t3, 0($t4)          # A[i] = B[i]

# i++
addi $s0, $s0, 1
j    FOR

ENDFOR:
# j = 0;
add  $s1, $zero, $zero
```

e.
C:
```
j = value;
max = 0;
for(i = 0; i < j; i++)
    if(A[i] > max) max = A[i];
j = 0;
```
MIPS:
```
# j = value;
add  $s1, $s5, $zero

# max = 0;
add  $s4, $zero, $zero

# i = 0;
add  $s0, $zero, $zero

FOR:
# i < j ?
slt  $t0, $s0, $s1
beq  $t0, $zero, ENDFOR

# lấy A[i]
sll  $t1, $s0, 2
add  $t2, $s3, $t1        # t2 = &A[i]
lw   $t3, 0($t2)          # t3 = A[i]

# if (A[i] > max)
slt  $t4, $s4, $t3        # t4 = 1 nếu max < A[i]
beq  $t4, $zero, SKIP

# max = A[i]
add  $s4, $t3, $zero

SKIP:
# i++
addi $s0, $s0, 1
j    FOR

ENDFOR:
# j = 0;
add  $s1, $zero, $zero
```